

FinPlanner - Current State Analysis

Executive Summary

FinPlanner is an existing personal finance management application built with Next.js 14, React 18, Prisma/PostgreSQL, and Tailwind CSS. The codebase is well-structured with ~200K+ lines of code and comprehensive features.

Current Architecture

Technology Stack

- **Frontend:** Next.js 14 (App Router), React 18, TypeScript 5
- **UI:** Tailwind CSS 3, Radix UI, shadcn/ui components
- **Backend:** Next.js API Routes
- **Database:** PostgreSQL 15 with Prisma 6 ORM
- **Authentication:** NextAuth.js with email/password

Database Schema (Existing)

The current Prisma schema includes:

Core Models

1. **User** - Authentication and user management
2. **Category** - Categorization for all financial entities
3. **Transaction** - All financial transactions (manual and parsed)
4. **Bill** - Recurring bills template
5. **BillInstance** - Individual occurrences of bills
6. **FinancialGoal** - Financial goals tracking
7. **GoalContribution** - Goal contributions with transaction linking
8. **Loan** - Loan tracking (home, personal, car, education, etc.)
9. **LoanPayment** - Individual loan payments
10. **Investment** - Investment portfolio items
11. **InvestmentTransaction** - Investment buy/sell transactions
12. **SIP** - Systematic Investment Plans
13. **Asset** - Physical and financial assets
14. **AIInsight** - AI-generated financial insights
15. **ChatConversation** & **ChatMessage** - AI chat functionality

Key Enumerations

- **TransactionType:** INCOME, EXPENSE, TRANSFER, INVESTMENT_BUY, INVESTMENT_SELL
- **TransactionSource:** MANUAL, SMS, EMAIL, BANK_STATEMENT, BILL
- **BillFrequency:** WEEKLY, MONTHLY, QUARTERLY, HALF_YEARLY, YEARLY, ONE_TIME
- **BillStatus:** PENDING, PAID, OVERDUE, CANCELLED
- **GoalType:** SAVINGS, DEBT_PAYOFF, INVESTMENT, EMERGENCY_FUND, etc.
- **LoanType:** HOME_LOAN, PERSONAL_LOAN, CAR_LOAN, EDUCATION_LOAN, etc.

Existing Modules & Features

1. Dashboard (/app/dashboard)

- **Status:**  Exists
- **Current Features:**
 - Financial overview cards
 - Quick actions bar
 - Goals progress widget
 - Loans status widget
 - Investment snapshot widget
 - Upcoming items widget
 - Category breakdown widget
- **Data Flow:** Redirects to /ai-home (AI-powered dashboard)

2. Transactions (/app/transactions)

- **Status:**  Exists
- **Current Features:**
 - Transaction listing with filtering
 - Manual transaction entry
 - SMS/Email parsing for automatic transaction creation
 - Category-wise analysis
 - Date range filtering
 - Transaction status tracking (SUCCESS, FAILED, PENDING)
- **API Routes:**
 - GET/POST /api/transactions - CRUD operations
 - POST /api/transactions/parse - SMS/Email parsing endpoint
 - GET /api/transactions/stats - Transaction statistics

3. Bills & Utilities (/app/bills)

- **Status:**  Comprehensive Implementation
- **Current Features:**
 - Bill templates (recurring bills)
 - Bill instances (individual occurrences)
 - Multiple views: Overview, Monthly View, Yearly View
 - Frequency support: WEEKLY, MONTHLY, QUARTERLY, HALF_YEARLY, YEARLY, ONE_TIME
 - Mark as paid/unpaid functionality
 - Bill analytics and statistics
 - Calendar navigation (month/year)
- **API Routes:**
 - GET/POST /api/bills - CRUD operations with view filtering
 - POST /api/bills/:id/payment - Mark bill as paid
 - DELETE /api/bills/:id/payment - Mark bill as unpaid
 - GET /api/bills/stats - Bill statistics
 - GET /api/bills/analytics - Analytics data
 - POST /api/bills/generate - Generate bill instances

Current Bills Implementation Analysis:

- **Bill Template** (Bill model): Stores recurring bill definition with `nextDueDate`
- **Bill Instances** (BillInstance model): Individual occurrences with status (PENDING, PAID, OVERDUE)
- **Payment Tracking**: Links to Transaction model when marked as paid
- **Due Date Management**: Has `nextDueDate` on Bill model but **no automatic rolling logic found**

4. Financial Goals (`/app/goals`)

- **Status:**  Exists
- **Current Features:**
 - Goal creation and tracking
 - Progress visualization
 - Goal contributions
 - Investment linking
 - Target amount and date tracking

5. Loans (`/app/loans`)

- **Status:**  Exists
- **Current Features:**
 - Loan tracking (multiple types)
 - EMI calculator
 - Payment history
 - Prepayment tracking

6. Investments (`/app/investments`)

- **Status:**  Comprehensive Implementation
- **Current Features:**
 - Portfolio tracking (14+ asset classes)
 - Investment transactions
 - SIP management
 - Platform tracking (Zerodha, Groww, etc.)
 - Goal linking (many-to-many)

7. SMS/Email Parsing

- **Status:**  Exists with Comprehensive Patterns
- **Location:** `/lib/transaction-parser.ts` , `/components/sms-parser-dialog.tsx`
- **Current Features:**
 - Pattern-based parsing for Indian banks (SBI, HDFC, ICICI, Axis)
 - UPI transaction parsing (PhonePe, Paytm, GPay, Amazon Pay, BHIM)
 - Extracts: amount, type, merchant, account, transaction ID, date, balance
 - Confidence-based suggestions
 - Review and edit before saving
- **API Route:** `POST /api/transactions/parse`

8. AI Features (`/app/ai-home`)

- **Status:**  Exists
- **Current Features:**
 - AI chat for financial queries
 - AI-generated insights

- Natural language transaction parsing
- Financial recommendations

Current Data Flow

```

User Interaction
  ↓
Frontend Component (Next.js Client Component)
  ↓
API Route Handler (/app/api/*)
  ↓
Prisma ORM
  ↓
PostgreSQL Database
  ↓
Response → Component Update → UI Render

```

Existing SMS Parsing Flow

```

User Pastes SMS → SMSParserDialog Component
  ↓
POST /api/transactions/parse
  ↓
TransactionParser.parse() (lib/transaction-parser.ts)
  ↓
Pattern matching + Category suggestion
  ↓
Return ParsedTransaction with confidence
  ↓
User Reviews/Edits in Dialog
  ↓
User Approves → POST /api/transactions
  ↓
Transaction Created in Database
  ↓
Dashboard/Analytics Updated

```

Gap Analysis for Required Features

✓ Already Implemented

1. **Bills Module:** Comprehensive with templates and instances
2. **SMS Parsing:** Robust pattern-based parser with review workflow
3. **Dashboard:** Exists with widgets and analytics
4. **Transaction Tracking:** Complete with categorization
5. **Multi-user Support:** Authentication with data isolation

⚠ Partially Implemented (Needs Extension)

1. **Bills as Renewals:**
 - ✓ Has frequency support (ONE_TIME, MONTHLY, QUARTERLY, HALF_YEARLY, YEARLY)
 - ✓ Has Bill and BillInstance models
 - ✓ Has mark paid functionality
 - ✗ **Missing:** Automatic due date rolling logic
 - ✗ **Missing:** Payment history with reference numbers/policy details

-  **Missing**: Reminder offsets (30/7/1 days before due)
-  **Missing**: Overdue/Due Soon status computation
-  **Missing**: Provider/vendor field

2. Dashboard Integration:

-  Has dashboard page
-  **Missing**: Month-to-date income/expenses/net (current calendar month)
-  **Missing**: Upcoming items list (next 30/60 days combining bills + renewals)
-  **Missing**: Quick action to add renewal

3. SMS Inbox Workflow:

-  Has SMS parser dialog
-  Has pattern matching
-  Has review/edit functionality
-  **Missing**: Batch SMS parsing (multiple SMS at once)
-  **Missing**: Inbox management (pending, approved, rejected states)
-  **Missing**: SMS storage and traceability
-  **Needs Enhancement**: Currently uses pattern matching, should integrate Abacus.AI LLM

Not Implemented

1. Global Command Bar:

- No global command bar/quick add functionality
- No natural language entry across all pages
- No LLM-based parsing for quick entries

2. Renewals as First-Class Module:

- Bills exist but don't fully match "Renewals/Planned Obligations" requirements
- Missing insurance-specific fields (policy number, provider, etc.)
- Missing reminder system

Design Decision: Extend Bills vs New Renewals Model

Option 1: Extend Existing Bills Module RECOMMENDED

Pros:

- Leverages existing infrastructure (Bill, BillInstance models)
- Already has frequency support matching requirements
- Has payment tracking via BillInstance
- Less code duplication
- Simpler migration path

Cons:

- Bills are conceptually recurring payments, might not fit all "obligation" types
- Need to add fields to existing models (migration required)

Required Changes:

1. Add fields to Bill model:

- `provider` (string, optional)
- `policyNumber` (string, optional)
- `reminderDays` (int array, e.g., [30, 7, 1])

2. Add fields to BillInstance model:

- `referenceNumber` (string, optional for payment reference)

- notes (already exists )
- 3. Implement automatic nextDueDate rolling logic
- 4. Implement overdue/due-soon status computation
- 5. Add reminder calculation utilities

Option 2: Create New Renewals Module

Pros:

- Clean separation of concerns
- Can design schema specifically for renewals/obligations
- No impact on existing Bills functionality

Cons:

- Code duplication (similar models and logic)
- Need to manage two similar but separate systems
- More complex “Upcoming Items” aggregation
- Larger codebase to maintain

Required Changes:

1. New models: Renewal, RenewalPayment
2. New API routes: /api/renewals
3. New UI pages: /app/renewals
4. Duplicate frequency logic and payment tracking
5. Complex aggregation for “Upcoming Items”

DECISION: Extend Bills Module

Rationale:

1. The existing Bill/BillInstance architecture already supports the core requirements
2. The frequency enums (ONE_TIME, MONTHLY, QUARTERLY, HALF_YEARLY, YEARLY) match exactly what's needed
3. The BillInstance model provides the payment history structure we need
4. Simpler to implement and maintain
5. Natural fit: renewals are a type of planned recurring obligation, which is what bills are

Implementation Approach:

1. **Schema Extension:** Add provider, policyNumber, reminderDays to Bill model
2. **Logic Enhancement:** Implement automatic date rolling when marking paid
3. **Status Computation:** Add utility functions for overdue/due-soon
4. **UI Enhancement:** Add fields to Bill forms and display
5. **API Enhancement:** Update API routes to support new fields and logic
6. **Rename/Rebrand:** Optionally rename “Bills” to “Bills & Renewals” in UI

Next Steps

Phase 1: Schema & Core Logic

1.  Update Prisma schema with new fields
2.  Create and run migrations
3.  Implement date rolling logic with tests
4.  Implement status computation utilities

Phase 2: API Enhancement

1. ☒ Update Bill API routes to handle new fields
2. ☒ Implement automatic date rolling on payment
3. ☒ Add upcoming obligations endpoint

Phase 3: UI Updates

1. ☒ Update Bill forms to include new fields
2. ☒ Update Bill display to show provider/policy
3. ☒ Add reminder indicators
4. ☒ Update dashboard with upcoming obligations

Phase 4: New Features

1. ☒ Implement global command bar with Abacus.AI LLM
2. ☒ Enhance SMS inbox with batch processing
3. ☒ Add month-to-date dashboard stats

Phase 5: Testing & Documentation

1. ☒ Write tests for date rolling logic
2. ☒ Add seed data for renewals
3. ☒ Update README with new features
4. ☒ Create design decisions document

Conclusion

The existing FinPlanner codebase is well-architected and comprehensive. The Bills module provides a solid foundation that can be extended to support the full Renewals/Planned Obligations requirements without creating a separate system. This approach will be more maintainable and leverage existing infrastructure effectively.